

BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN
MÔN HỌC: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI: PHÂN ĐOẠN KHỐI U NÃO

Giảng viên hướng dẫn: TS.Nguyễn Thị Mai Trang

Nhóm lớp: 14

Nhóm bài tập lớn: 10

Phạm Trịnh Đức
Luu Minh Hiền
Phạm Quốc Anh

B22DCCN242
B22DCCN290
B22KHCCN006

Hà Nội, 2025

LỜI CẢM ƠN

Trước tiên, nhóm em xin được bày tỏ lòng biết ơn chân thành đến các thầy cô trong Học viện Công nghệ Bru chính Viễn thông, đặc biệt là các giảng viên khoa Công nghệ Thông tin I – những người đã tận tình giảng dạy, truyền đạt kiến thức và tạo nền tảng vững chắc cho quá trình học tập, nghiên cứu của chúng em.

Nhóm em xin gửi lời cảm ơn sâu sắc đến cô Nguyễn Thị Mai Trang – giảng viên hướng dẫn, người đã luôn tận tâm hỗ trợ, định hướng và đưa ra những góp ý quý báu trong suốt quá trình thực hiện đề tài. Sự giúp đỡ và động viên của cô là nguồn động lực lớn giúp nhóm hoàn thành đồ án đúng tiến độ.

Nhóm cũng xin chân thành cảm ơn gia đình, bạn bè và các anh chị khóa trên – những người đã luôn đồng hành, động viên và hỗ trợ nhóm trong suốt quá trình làm việc và học tập.

Mặc dù nhóm đã nỗ lực hoàn thành tốt nhất trong khả năng, tuy nhiên do hạn chế về thời gian và kinh nghiệm thực tế, đồ án không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những nhận xét, góp ý quý báu từ quý thầy cô để hoàn thiện hơn trong những lần nghiên cứu tiếp theo.

Xin trân trọng cảm ơn!

MỤC LỤC

DANH MỤC HÌNH ẢNH	3
I. GIỚI THIỆU BÀI TOÁN.....	4
1. Bối cảnh nghiên cứu.....	4
2. Vấn đề nghiên cứu.....	4
3. Mục tiêu.....	4
II. CƠ SỞ LÝ THUYẾT.....	6
1. Lý thuyết tổng quan.....	6
1.1. Giới thiệu về bài toán segmentation	6
1.2. Ảnh MRI	8
1.3. Tổng quan về mô hình U-net	10
1.4. Mô hình Unet-3D	12
2. Giải quyết bài toán	12
2.1. Dữ liệu	12
2.2. Kiến trúc mô hình	17
2.3. Quy trình huấn luyện	19
2.3.1 Hàm loss.....	19
2.3.2 Evaluation	20
2.3.3 Optimizer	21
2.3.4 Quy trình training tổng quát.....	23
III. THỰC NGHIỆM	24
1. Môi trường thực nghiệm	24
2. Phân tích mã nguồn.....	24
2.1 Xử lý dữ liệu	24
2.2. Xây dựng mô hình	25
2.3 Huấn luyện mô hình.....	27
2.3.1 Training.....	27
2.3.2 Validation.....	28
3.Kết quả	30
4. Triển khai hệ thống	31
5. Phân tích.....	32
KẾT LUẬN.....	33
TÀI LIỆU THAM KHẢO	33

DANH MỤC HÌNH ẢNH

Figure 1: Mô tả bài toán Semantic Segmentation	6
Figure 2: Mô tả bài toán Instance Segmentation	7
Figure 3: Mô tả bài toán Panopic Segmentation	7
Figure 4: Hình ảnh mô tả khối não 3D và một lát cắt.	8
Figure 5: Các chế độ được hiển thị bởi một lát cắt trên chiều y	9
Figure 6: Kiến trúc tổng thể của mô hình Unet-2D.....	10
Figure 7: Kiến trúc tổng quát của mô hình Unet3D	12
Figure 8: Mô tả label của bài toán	13
Figure 9: Mô tả ảnh não 3D theo chiều Axial - Z	14
Figure 10: Mô tả ảnh não 3D theo chiều Sagittal - X.....	14
Figure 11: Mô tả ảnh não 3D theo chiều Coronal - Y	15
Figure 12: Quy trình xử lý dữ liệu.....	16
Figure 13: Kiến trúc tổng quát của mô hình.....	17
Figure 14: Chi tiết các lớp của mô hình	18
Figure 15: Quy trình huấn luyện mô hình	23
Figure 16: Quá trình xử lý dữ liệu.....	25
Figure 17: Xây dựng mô hình Unet3D.....	27
Figure 18: Hàm training cho một epoch.....	28
Figure 19: Hàm validation cho một epoch	29
Figure 20: Kết quả sau khi huấn luyện.....	30
Figure 21: Kết quả sau khi mô hình được huấn luyện.....	31
Figure 22: Kiến trúc hệ thống.....	32
Figure 23: Giao diện của hệ thống	32

I. GIỚI THIỆU BÀI TOÁN

1. Bối cảnh nghiên cứu

Khối u não là một trong những bệnh lý nghiêm trọng, ảnh hưởng đến hàng triệu người trên thế giới. Việc phát hiện và phân đoạn chính xác khối u từ ảnh chụp cộng hưởng từ (MRI) đóng vai trò quan trọng trong việc chẩn đoán và lập kế hoạch điều trị. Tuy nhiên, do kích thước, hình dạng và vị trí của khối u rất đa dạng, việc phân đoạn thủ công không gây ra khó khăn, tiêu tốn nhiều thời gian mà còn gây ra sai sót trong quá trình thực hiện - một điều cần hạn chế tối thiểu trong ngành Y khoa.

Nhiều khu vực trên thế giới, đặc biệt là các nước còn lạc hậu, nghèo nàn có thể kể đến là Châu Phi, việc tiếp cận dịch vụ chăm sóc sức khỏe vẫn là thách thức lớn trong năm 2024. Theo báo cáo của NCBI, tỉ lệ bác sĩ - bệnh nhân tại Uganda khoảng 1:25.000, trong khi ý tá - bệnh nhân khoảng 1:11.000 [source] thấp hơn nhiều so với mức khuyến nghị của WHO. Điều này gây ra sự chậm trễ nghiêm trọng trong chẩn đoán và điều trị bệnh, đặc biệt là các bệnh lý nguy hiểm như khối u não. Bệnh nhân phải chờ hàng tuần hoặc thậm chí hàng tháng để nhận kết quả chụp MRI, từ đó làm trì hoãn các can thiệp y tế quan trọng, bao gồm điều trị sau phẫu thuật hoặc xạ trị kịp thời.

Sự phát triển của trí tuệ nhân tạo (AI), đặc biệt là học sâu (Deep Learning) đã mang đến những giải pháp tự động hóa mang lại độ chính xác cao và giảm tải công việc cho con người. Trong đó, kiến trúc U-Net, được giới thiệu bởi Ronneberger và các cộng sự vào năm 2015, đã trở thành một tiêu chuẩn vàng trong phân đoạn ảnh y tế nhờ khả năng kết hợp thông tin ngữ cảnh và chi tiết cục bộ. Tuy nhiên, với dữ liệu 3D phức tạp như ảnh MRI khối u não, U-Net truyền thống vẫn còn những hạn chế về hiệu suất và khả năng tổng quát hóa.

2. Vấn đề nghiên cứu

Trong những năm gần đây, các phương pháp học sâu (deep learning) đã đạt được những bước tiến đáng kể trong lĩnh vực phân đoạn ảnh y tế, đặc biệt là trong việc phân tích hình ảnh cộng hưởng từ (MRI) của não bộ. Tuy nhiên, khi áp dụng vào bài toán phân đoạn khối u não từ dữ liệu thể tích (volumetric data), vẫn còn tồn tại nhiều thách thức cần được giải quyết.

Khối u não có đặc điểm hình thái rất phức tạp: hình dạng bất quy tắc, kích thước thay đổi lớn giữa các bệnh nhân, và ranh giới mờ nhạt với các mô não bình thường. Điều này khiến việc phân đoạn trở nên khó khăn ngay cả với các chuyên gia y tế, chưa kể đến các mô hình tự động. Các phương pháp truyền thống như U-Net 2D khi được áp dụng cho dữ liệu 3D thường xử lý từng lát cắt MRI độc

lập, dẫn đến việc bỏ qua mối liên hệ không gian giữa các lát cắt, gây mất mát thông tin quan trọng trong cấu trúc thể tích của não bộ.

Mặc dù các mô hình U-Net 3D đã được phát triển nhằm giải quyết vấn đề trên bằng cách trực tiếp xử lý các khối dữ liệu 3D, chúng vẫn gặp khó khăn trong việc học và biểu diễn các đặc trưng không gian phức tạp, đặc biệt là trong việc phát hiện các vùng khối u nhỏ hoặc có ranh giới mờ. Ngoài ra, việc xử lý dữ liệu y tế 3D đòi hỏi tài nguyên tính toán lớn, làm gia tăng độ phức tạp trong triển khai thực tế.

Do đó, nghiên cứu và cải tiến mô hình U-Net 3D là một hướng tiếp cận quan trọng để nâng cao chất lượng phân đoạn ảnh y tế trong bối cảnh thực tế lâm sàng, đồng thời tối ưu hiệu quả xử lý trên tập dữ liệu thể tích.

3. Mục tiêu

Mục tiêu chính của báo cáo này là nghiên cứu, cài đặt và đánh giá mô hình U-Net 3D trong bài toán phân đoạn khối u não từ ảnh MRI. Cụ thể, nhóm nghiên cứu tập trung vào việc kiểm tra khả năng của mô hình trong việc phát hiện chính xác các vùng khối u với nhiều đặc điểm hình thái khác nhau, từ đó đánh giá hiệu quả tổng thể của mô hình thông qua các chỉ số đánh giá như Dice Score, Intersection over Union (IoU), và Accuracy.

Bên cạnh đó, báo cáo cũng tiến hành so sánh hiệu suất của U-Net 3D với các mô hình cơ bản hơn như U-Net 2D truyền thống, nhằm chỉ ra lợi ích của việc xử lý trực tiếp dữ liệu thể tích 3D trong bối cảnh y tế. Kết quả từ nghiên cứu này kỳ vọng sẽ làm rõ tiềm năng của mô hình U-Net 3D như một công cụ hỗ trợ chẩn đoán, từ đó góp phần cải thiện chất lượng và tốc độ phát hiện khối u não trong thực hành lâm sàng.

Thông qua nghiên cứu, nhóm mong muốn đóng góp một mô hình AI có độ chính xác cao, dễ triển khai, và phù hợp với nhu cầu thực tế trong lĩnh vực y tế hiện đại, nơi việc phát hiện sớm và chính xác khối u có vai trò then chốt trong việc điều trị và cải thiện tiên lượng cho bệnh nhân.

II. CƠ SỞ LÝ THUYẾT

1. Lý thuyết tổng quan

1.1. Giới thiệu về bài toán segmentation

Phân đoạn ảnh (image segmentation) là một bài toán quan trọng trong thị giác máy tính, nhằm chia một bức ảnh thành các vùng có ý nghĩa riêng biệt, mỗi vùng tương ứng với một đối tượng hoặc thành phần cụ thể. Trong phân đoạn ngữ nghĩa (semantic segmentation), mỗi pixel trong ảnh được gán một nhãn lớp (class label), ví dụ: "khối u," "mô lành," hoặc "nền." Bài toán này đặc biệt quan trọng trong các ứng dụng y tế, nơi việc xác định chính xác ranh giới của các cấu trúc giải phẫu (như khối u hoặc cơ quan) có thể ảnh hưởng trực tiếp đến chẩn đoán và điều trị. Ngoài ra bài toán cũng được ứng dụng trong nhiều các ứng dụng thực tiễn khác

Phân đoạn ảnh được chia thành ba loại chính:

- Phân đoạn ngữ nghĩa (Semantic Segmentation): Gán nhãn cho từng pixel thuộc một lớp cụ thể mà không phân biệt các đối tượng riêng lẻ.

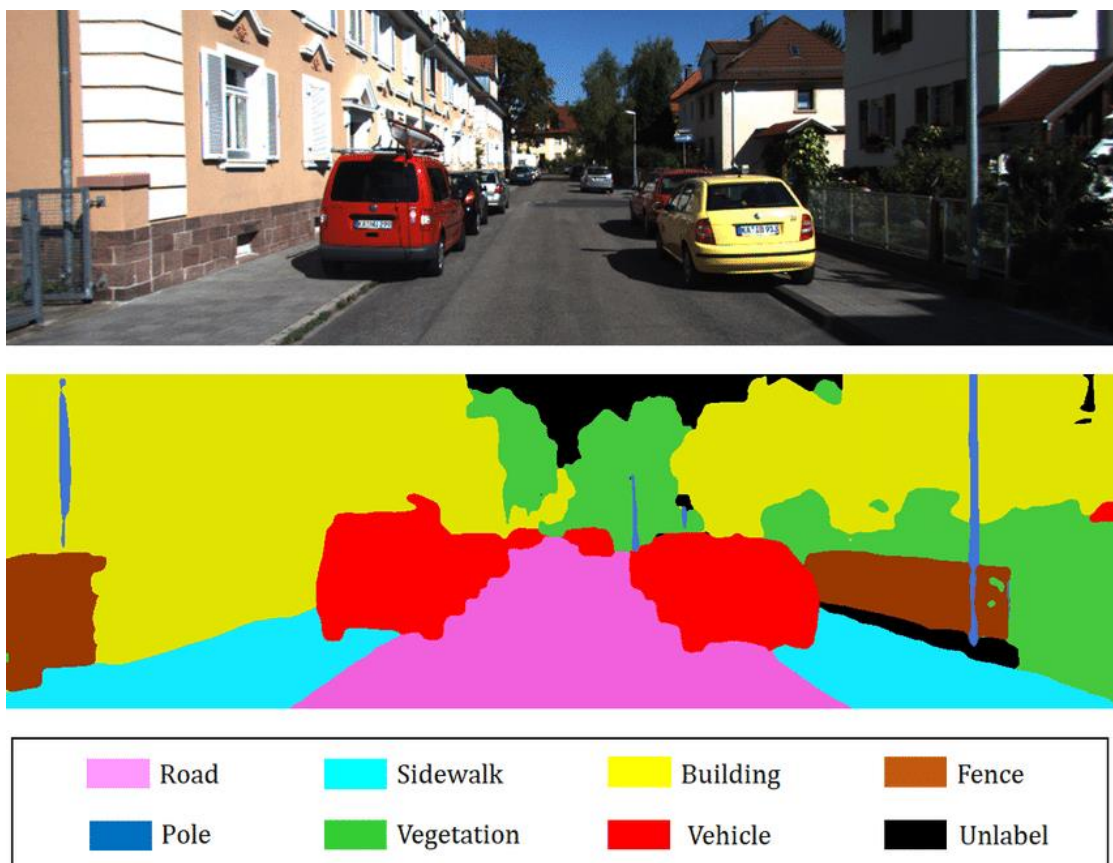


Figure 1: Mô tả bài toán Semantic Segmentation

- Phân đoạn thực thể (Instance Segmentation): Không chỉ gán nhãn lớp mà còn phân biệt các đối tượng riêng lẻ thuộc cùng một lớp.



Figure 2: Mô tả bài toán Instance Segmentation

- Phân đoạn toàn cảnh (Panoptic Segmentation): Kết hợp cả phân đoạn ngữ nghĩa và thực thể.



Figure 3: Mô tả bài toán Panopic Segmentation

Trong bài toán phân đoạn khối u não, chúng ta tập trung vào phân đoạn ngữ nghĩa (Semantic segmentation), với mục tiêu xác định các vùng khối u (bao gồm u não lành tính và ác tính), mô lành, và các cấu trúc khác từ ảnh MRI.

1.2. Ảnh MRI

Phân đoạn ảnh y tế là một lĩnh vực chuyên biệt của image segmentation, tập trung vào việc phân tích các hình ảnh y khoa như MRI, CT, hoặc X-quang. Mục tiêu là xác định các cấu trúc giải phẫu hoặc bệnh lý (như khối u, mô tổn thương,

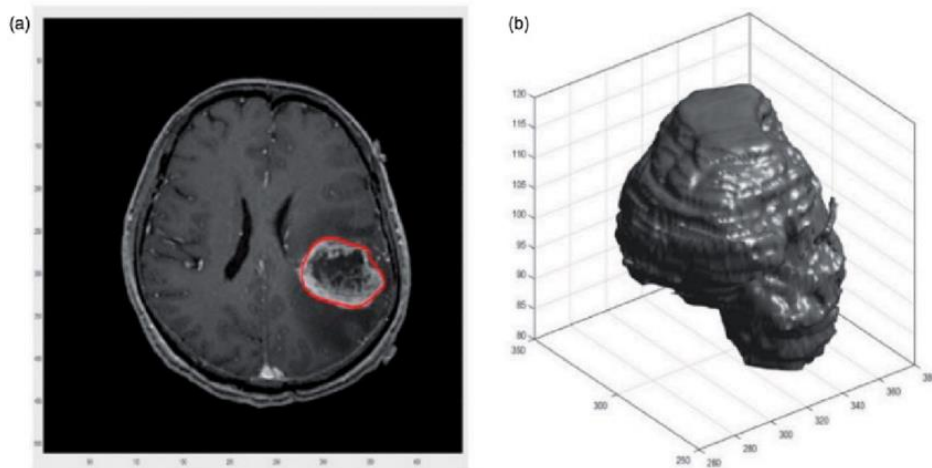


Figure 4: Hình ảnh mô tả khối não 3D và một lát cắt.

hoặc cơ quan) với độ chính xác cao. Trong trường hợp khối u não, ảnh MRI cung cấp dữ liệu thể tích 3D, thường bao gồm các chế độ (modalities) như T1, T1ce (T1 với chất tương phản), T2, và FLAIR, mỗi loại cung cấp thông tin khác nhau về mô não và khối u.

a. Native (T1)

- Đây là ảnh chụp MRI theo chuỗi T1-weighted (T1W).
- T1W giúp hiển thị rõ các cấu trúc giải phẫu của não, với dịch não tủy (CSF) có màu tối, chất xám có màu xám và chất trắng có màu sáng.
- Được sử dụng để phân biệt mô não bình thường và bất thường.

b. Post-contrast T1-weighted (T1CE)

- Đây là ảnh T1-weighted có sử dụng chất tương phản (Contrast-Enhanced, CE), thường là Gadolinium.
- Chất tương phản giúp tăng độ sáng của các vùng mô bất thường, đặc biệt là các khối u hoặc vùng có hàng rào máu não bị tổn thương.
- Thường được sử dụng để xác định vị trí và kích thước khối u.

c. T2-weighted (T2)

- Ảnh T2-weighted (T2W) làm nổi bật sự khác biệt giữa các mô bằng cách tăng cường độ sáng của dịch não tủy (CSF), trong khi chất trắng xuất hiện tối hơn chất xám.
- Rất hữu ích để phát hiện các bất thường như phù nề xung quanh khối u, tổn thương hoặc viêm.

d. T2 Fluid Attenuated Inversion Recovery (FLAIR)

- Đây là một biến thể của ảnh T2, trong đó tín hiệu từ dịch não tủy (CSF) được triệt tiêu, giúp làm rõ các tổn thương nằm gần hoặc bên trong não thất.
- Được sử dụng phổ biến để phát hiện phù nề, tổn thương viêm nhiễm hoặc bệnh lý não trắng (ví dụ: đa xơ cứng - MS).

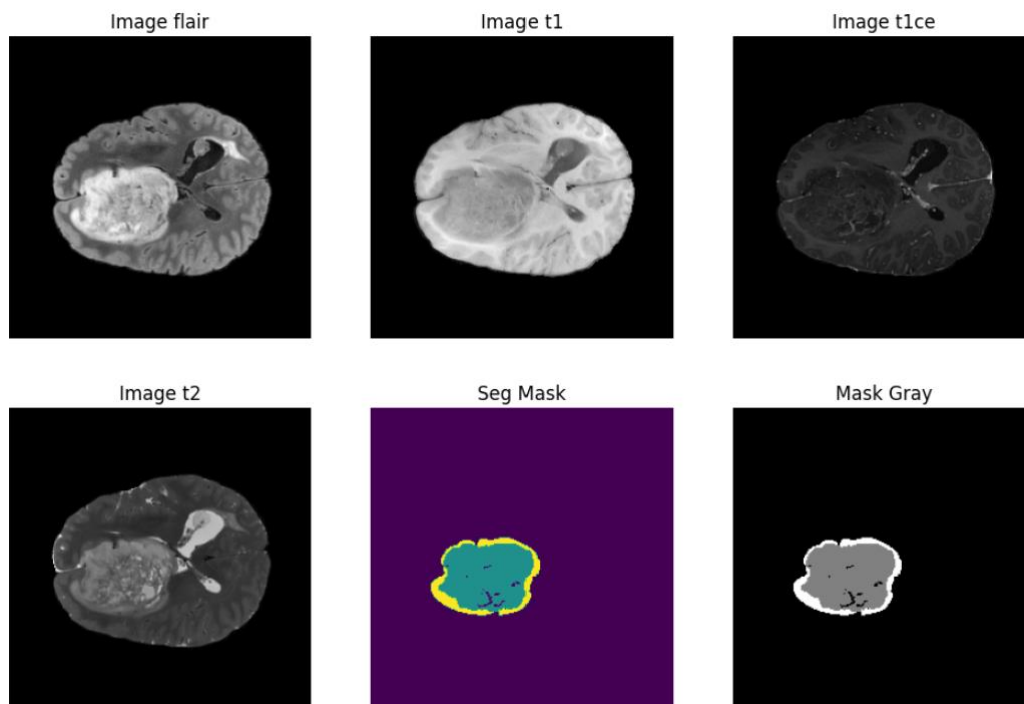


Figure 5: Các chế độ được hiển thị bởi một lát cắt trên chiều y

Các thách thức chính trong phân đoạn ảnh y tế bao gồm:

- Độ phức tạp của dữ liệu: Dữ liệu y tế thường có kích thước lớn, trung bình mỗi ảnh MRI (3D) có dung lượng lên tới 64MB lớn hơn rất nhiều so với 1 ảnh 2D thông thường và chứa nhiễu.
- Sự đa dạng của đối tượng: Khối u não có hình dạng, kích thước, và vị trí không đồng nhất.
- Ranh giới không rõ ràng: Khối u thường hòa lẫn với mô xung quanh, gây khó khăn trong việc xác định ranh giới.

- Thiếu dữ liệu huấn luyện: Dữ liệu y tế thường bị hạn chế do vấn đề bảo mật và chi phí thu thập. Đặc biệt cần có các chuyên gia về y tế mới có thể gán nhãn được.

Để giải quyết các thách thức này, các phương pháp học sâu như U-Net và các biến thể của nó đã được sử dụng rộng rãi, kết hợp với các kỹ thuật tiền xử lý dữ liệu và tăng cường dữ liệu (augmentation).

1.3. Tổng quan về mô hình U-net

- U-Net là một mô hình deep learning phổ biến dùng trong image segmentation, đặc biệt là trong lĩnh vực y tế (như phân đoạn khối u não, mô phổi, v.v.).
- Được giới thiệu bởi Olaf Ronneberger vào năm 2015, U-Net đã trở thành tiêu chuẩn trong Medical Image Segmentation do khả năng học hiệu quả từ dữ liệu ít và cho ra kết quả tốt.
- Paper gốc: "U-Net: Convolutional Networks for Biomedical Image Segmentation"

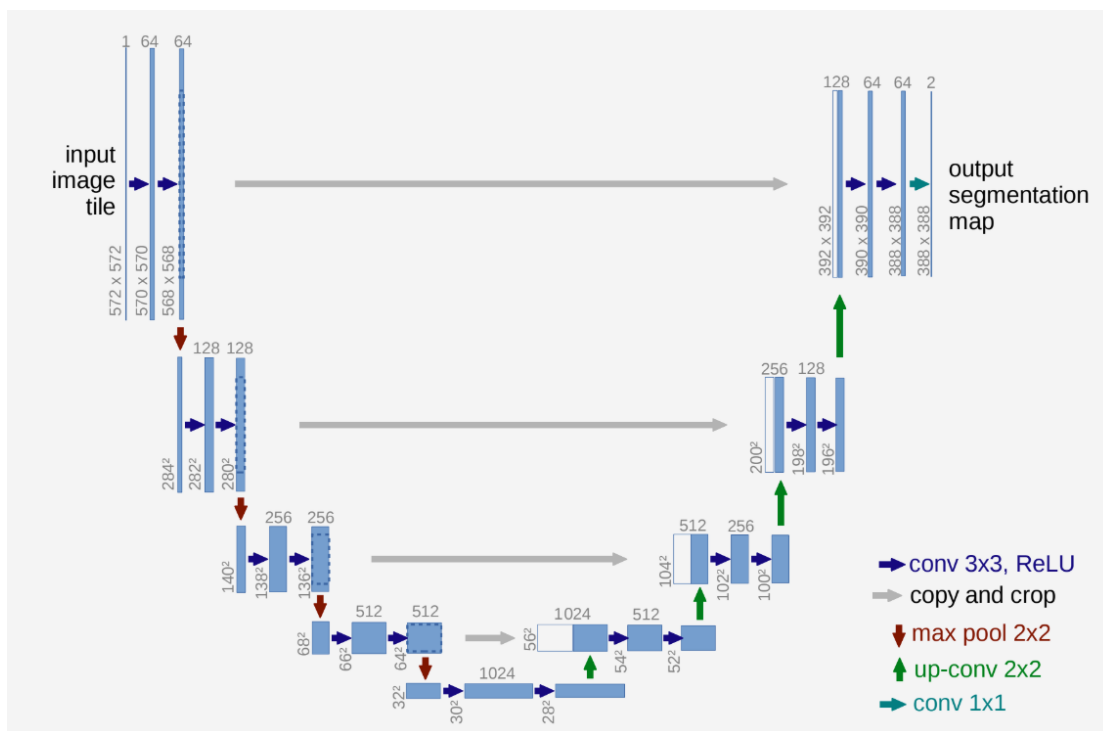


Figure 6: Kiến trúc tổng thể của mô hình Unet-2D

- Kiến trúc của Unet và các điểm chính của mô hình
Unet có dạng hình chữ U gồm 2 thành phần chính:
 - a. Encoder (Contraction Path) sử dụng các lớp Convolution + ReLU + Maxpooling:
 - Trích xuất đặc trưng ảnh đầu vào:
 - Giảm kích thước không gian (Spatial size) của ảnh nhưng tăng chiều (channels) để nắm bắt thông tin ở mức trừu tượng

- Loại bỏ nhiễu không quan trọng, chỉ giữ lại những đặc trưng cần thiết cho segmentation
 - Lợi ích:
 - Giúp mô hình hiểu ngữ cảnh toàn cục của ảnh
 - Giúp phân biệt giữa các đối tượng nền.
- b. Decoder (Expansion Path) sử dụng Upsampling + Convolution:
- Khôi phục lại kích thước ảnh về như ban đầu
 - Kết nối thông tin từ Encoder với các đặc trưng chi tiết hơn
 - Dự đoán (class cho từng pixel)
 - Lợi ích:
 - Phục hồi chi tiết hình ảnh để đạt segmentation chính xác
 - Kết hợp Encoder để giữ lại ngữ cảnh về thông tin vị trí.
- c. Các skip connections nối trực tiếp các feature từ Encoder → Decoder
- Giữ lại thông tin chi tiết từ ảnh gốc
 - Giảm mất mát thông tin do MaxPooling
 - Tăng độ chính xác khi phân đoạn các biên vật thể nhỏ
 - Lợi ích:
 - Giúp mô hình phân đoạn được các đối tượng nhỏ hoặc biên phức tạp
 - Giải quyết vấn đề vanishing gradient, giúp mô hình hội tụ tốt hơn.
- d. Kết hợp cả thông tin cục bộ và toàn cục
- Encoder học thông tin toàn cục (bằng cách giảm kích thước ảnh dần dần).
 - Decoder học thông tin cục bộ (bằng cách phục hồi thông tin chi tiết).
 - Skip connections giúp kết hợp cả hai loại thông tin này.
 - Lợi ích:
 - Giúp mô hình vừa nhận diện được cấu trúc tổng thể, vừa bảo toàn được chi tiết trong ảnh.
 - Giảm lỗi segmentation ở các vùng phức tạp
- e. U-Net hoạt động tốt với dữ liệu ít
- Do có skip connections và kiến trúc cân đối, U-Net có thể hoạt động tốt ngay cả khi không có nhiều dữ liệu huấn luyện.
 - Các mô hình segmentation khác như FCN, DeepLab thường yêu cầu tập dữ liệu lớn hơn.
 - Lợi ích: Rất phù hợp cho các bài toán y tế, nơi dữ liệu thường bị hạn chế.
- f. Các biến thể của U-Net

- 3D U-Net → Dùng cho dữ liệu ảnh y tế 3D.
- Attention U-Net → Thêm Attention Mechanism để tập trung vào vùng quan trọng.
- U-Net++ → Thêm nhiều skip connections và kiến trúc phức tạp hơn.
- Swin U-Net → Kết hợp Transformer để cải thiện khả năng nhận dạng vùng cần segment.

g. Ứng dụng của U-Net

- U-Net đặc biệt mạnh trong các bài toán image segmentation như:
 - Phân đoạn khối u não từ MRI (BraTS Challenge)
 - Phân vùng phổi từ ảnh CT
 - Phát hiện tổn thương võng mạc từ ảnh fundus
 - Giám sát môi trường
 - Phân đoạn tế bào trong sinh thiết mô
 - Phát hiện đường biên vật thể trong ảnh vệ tinh

1.4. Mô hình Unet-3D

Mô hình Unet-3D có kiến trúc tương tự mô hình Unet-2D bên trên, tuy nhiên mô hình này sử dụng các Convolution 3D thay vì Convolution 2D. Convolution 3D giúp xử lý ảnh 3D dễ dàng và hiệu quả hơn.

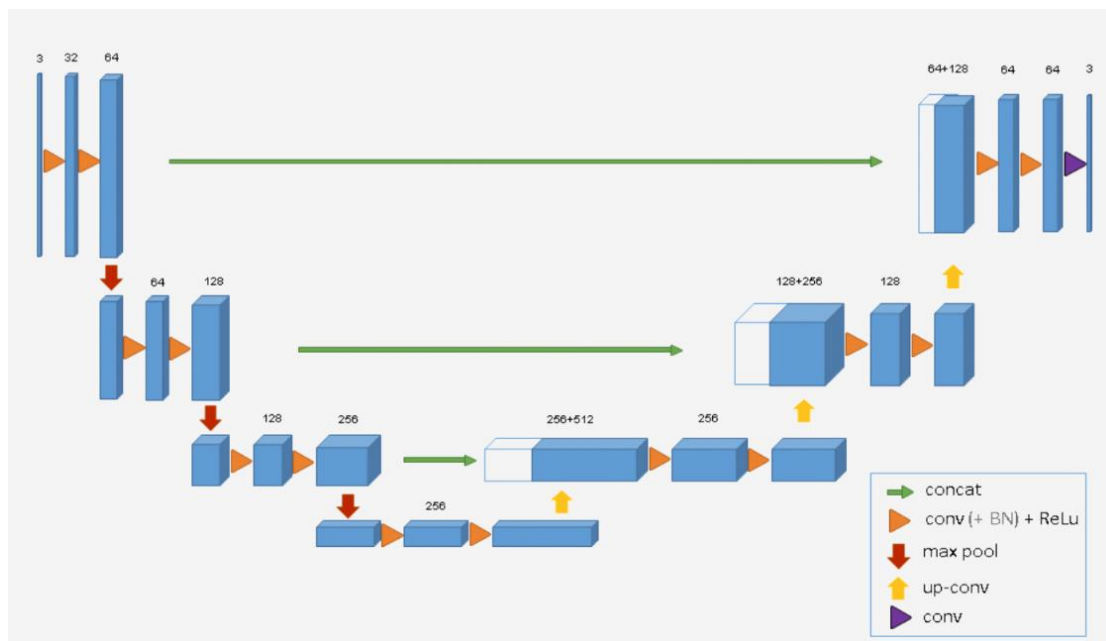


Figure 7: Kiến trúc tổng quát của mô hình Unet3D

2. Giải quyết bài toán

2.1. Dữ liệu

Một số thông tin về tập dữ liệu BraTS2023

- BraTS (Brain Tumor Segmentation) là một tập dữ liệu y khoa được sử dụng phổ biến trong nghiên cứu về chẩn đoán và phân đoạn khối u não. Dữ liệu này chứa hình ảnh chụp cộng hưởng từ (MRI) với nhiều chế độ khác nhau, có dung lượng lên tới 70-80 GB.

- Cấu trúc dataset

Dữ liệu chứa trong tập brats2023-part-1 (625 sample) có cấu trúc:

```
brats2023-part-1
├── BraTS-GLI-00000-000
├── BraTS-GLI-00002-000
├── BraTS-GLI-00003-000
├── BraTS-GLI-00005-000
...
```

Mỗi sample có cấu trúc:

```
brats2023-part-1/BraTS-GLI-00000-000
├── BraTS-GLI-00000-000-seg.nii
├── BraTS-GLI-00000-000-t1c.nii
├── BraTS-GLI-00000-000-t1n.nii
├── BraTS-GLI-00000-000-t2f.nii
└── BraTS-GLI-00000-000-t2w.nii
```

Mỗi file *.nii* là 1 ảnh 3D, bốn file tương ứng với bốn ảnh được chụp theo bốn chế độ: T1, T1CE, T2, FLAIR. Với file *seg.nii* chứa ảnh được phân đoạn theo bốn classes:

Class Index	Description	Viridis Color (for visualization)
0	Background	Dark purple (or black)
1	Necrotic and Non-enhancing Tumor Core	Blue or dark green
2	Peritumoral Edema/Invaded Tissue	Light green or teal
3	GD-Enhancing Tumor	Yellow

Figure 8: Mô tả label của bài toán

- Trực quan dataset:

Hiển thị ảnh não 3D theo trục từ trên xuống (Axial - Z)

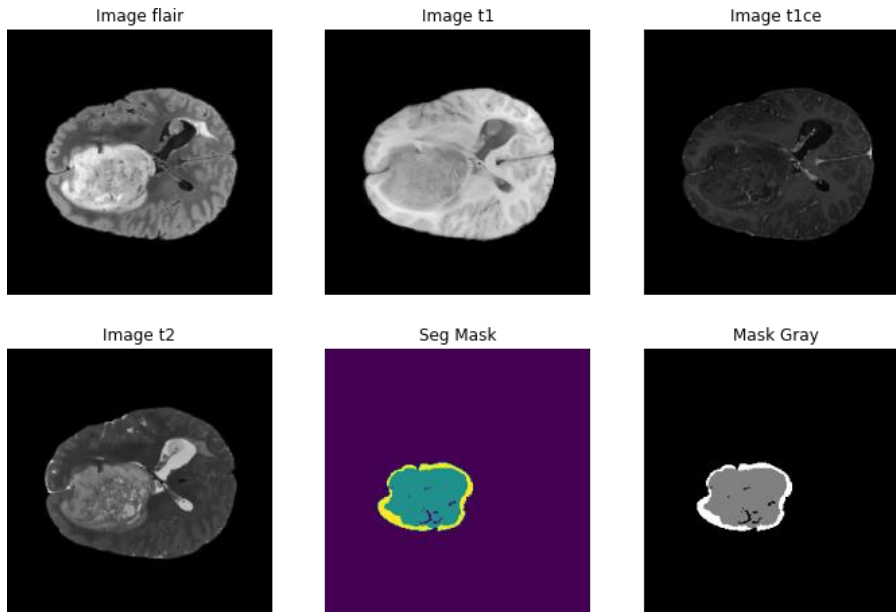


Figure 9: Mô tả ảnh não 3D theo chiều Axial-Z

Plot ảnh 3D theo chiều từ trái qua phải (Sagittal - X)

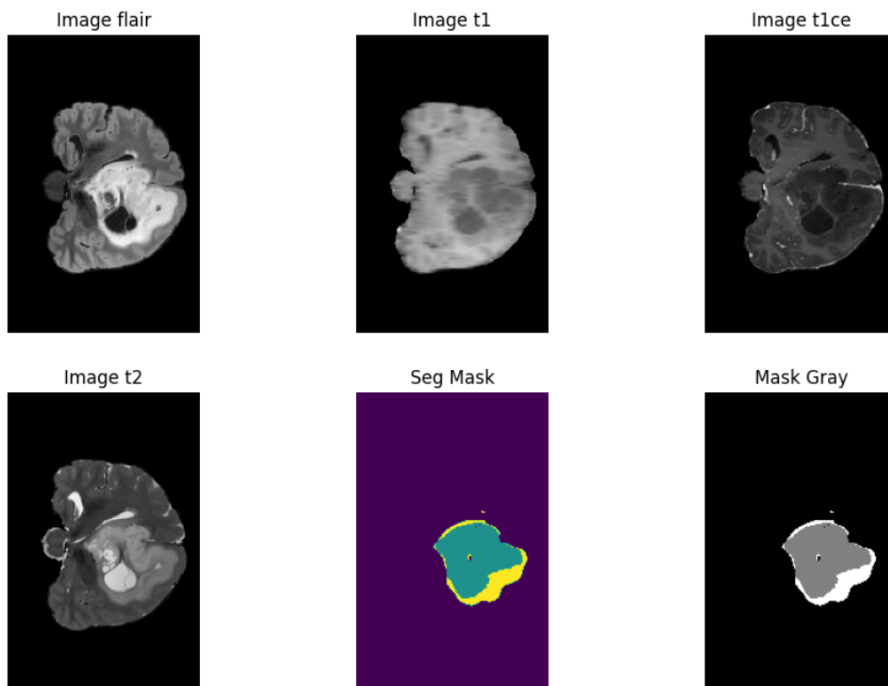


Figure 10: Mô tả ảnh não 3D theo chiều Sagittal - X

Plot ảnh 3D theo chiều từ trước ra sau (Coronal - Y)

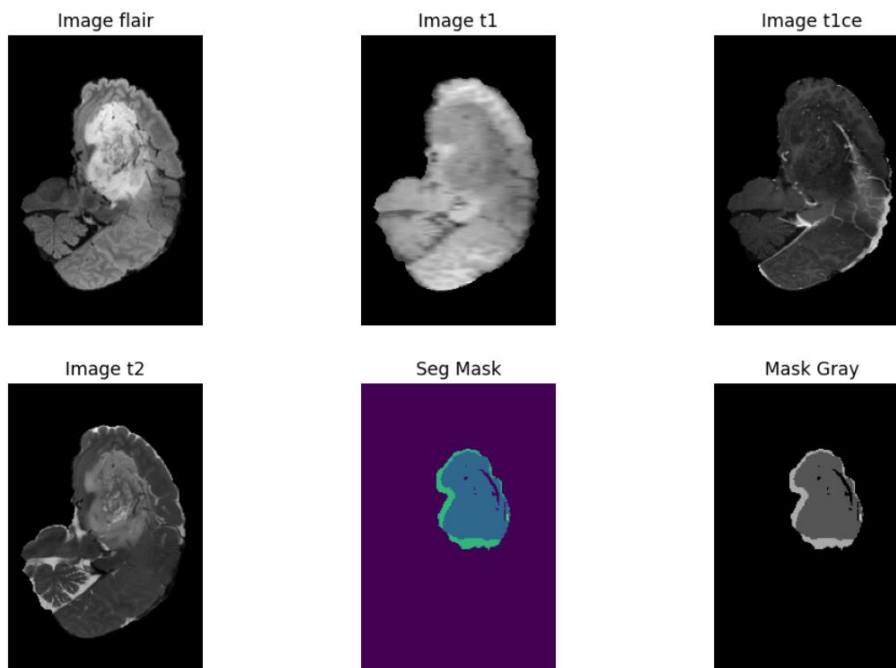


Figure 11: Mô tả ảnh não 3D theo chiều Coronal - Y

- Xử lý dữ liệu:
 - Huấn luyện với kích thước ảnh gốc là tốn bộ nhớ:
Nếu sử dụng toàn bộ ảnh gốc (với đầy đủ kích thước ban đầu) để huấn luyện mô hình, điều này sẽ yêu cầu một lượng bộ nhớ rất lớn và có thể không cần thiết vì không phải toàn bộ phần ảnh đều chứa thông tin quan trọng cho việc phân tích (chẳng hạn như các vùng ngoài não không chứa thông tin về bệnh lý). Vì vậy ta sẽ chỉ lấy ba loại ảnh được cho là giàu thông tin hơn là Flair, TCE, T2 và hợp nhất chúng lại
 - Tập trung vào ROI (Region of Interest – Vùng quan tâm):
Thay vì huấn luyện trên toàn bộ ảnh, một cách tiếp cận tối ưu là chỉ lấy phần ảnh chứa vùng não (vùng có nhiều thông tin hữu ích về bệnh lý) để giảm thiểu dữ liệu không cần thiết. ROI ở đây đề cập đến phần ảnh quanh vùng não, nơi mà hầu hết các đặc điểm cần thiết để chẩn đoán hoặc phân đoạn nằm.
 - Việc cắt xén (slicing và cropping) từ 56 đến 184:
Qua quá trình thử nghiệm, đã xác định rằng việc lấy phần ảnh (theo chiều thứ 3 – thường là trục z, chứa các lát cắt của ảnh 3D) từ chỉ số 56 đến 184 tạo thành một ROI hợp lý. Nghĩa là, trong số các lát cắt của ảnh, các lát từ thứ 56 đến 184 chứa phần não cần thiết cho bài toán, và các lát cắt bên ngoài khoảng này có thể không quan trọng hoặc chứa nhiều thông tin dư thừa.
- Chuẩn bị dữ liệu:

- Bỏ qua việc đã có phần cứng đầy đủ, ở đây ta chỉ sử dụng Google Colab hoặc Kaggle. Hai nền tảng cung cấp phần cứng giúp việc training mô hình Deep Learning dễ dàng hơn. Tuy nhiên ở đây ta sẽ không sử dụng được Google Colab vì ổ cứng không đủ để chứa dữ liệu, vì vậy ta sẽ tận dụng kho lưu trữ dữ liệu của Kaggle để lưu trữ và tải dữ liệu. Quy trình xử lý dữ liệu như sau:

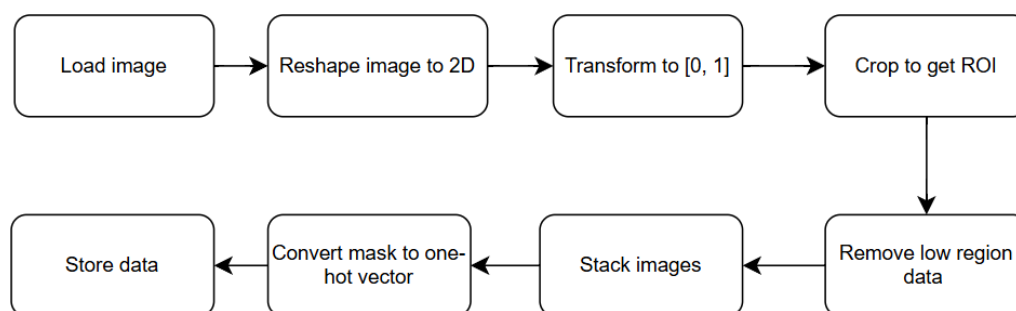


Figure 12: Quy trình xử lý dữ liệu

- Ở đây có một vấn đề cần lưu ý, ổ cứng của Kaggle chỉ có 20GB, không đủ lưu trữ toàn bộ 70-80GB dữ liệu. Vì vậy, ta sẽ tận dụng kho lưu trữ dữ liệu của Kaggle cho phép người dùng tải 200GB dữ liệu. Ta sẽ lưu từng tập nhỏ của dữ liệu sau đó tải từng tập nhỏ dữ liệu lên, chi tiết cách code sẽ có trong repository ở bên dưới.

2.2. Kiến trúc mô hình

- Kiến trúc tổng quát

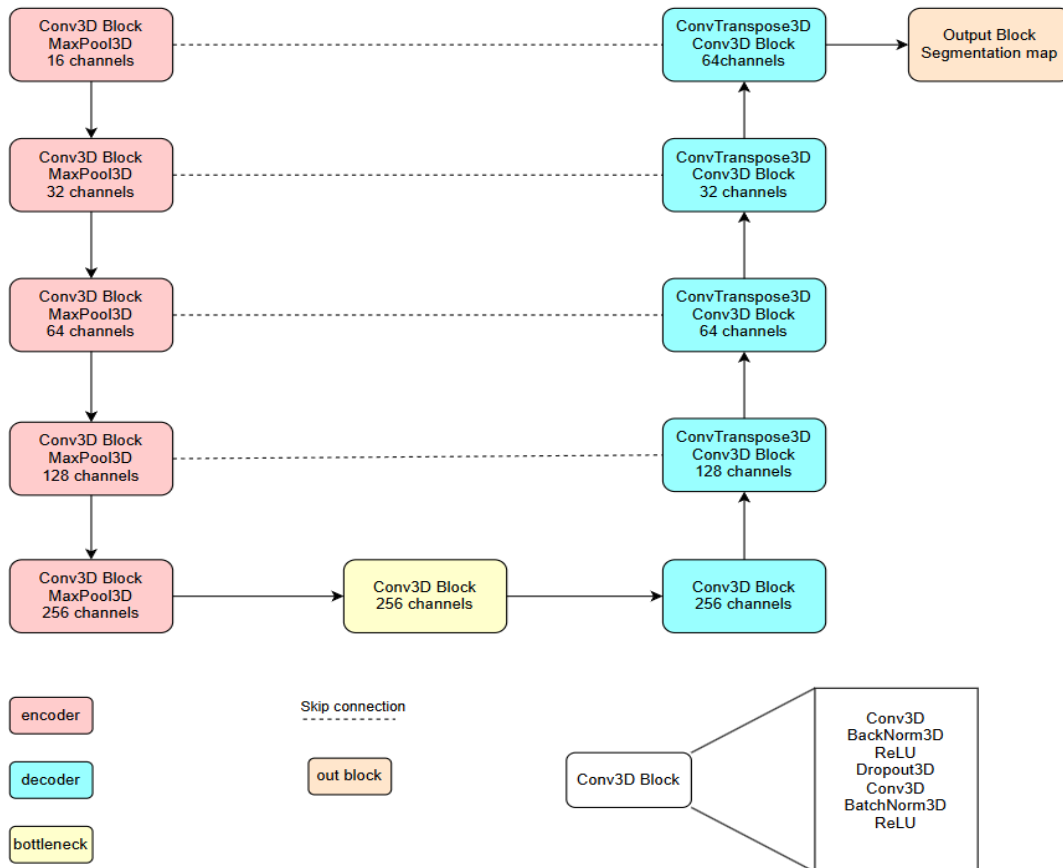


Figure 13: Kiến trúc tổng quát của mô hình

- Kiến trúc chi tiết của mô hình:

Layer (type:depth-idx)	Param #
UNETR	--
Sequential: 1-1	--
Conv3d: 2-1	1,312
BatchNorm3d: 2-2	32
ReLU: 2-3	--
Dropout: 2-4	--
Conv3d: 2-5	6,928
BatchNorm3d: 2-6	32
ReLU: 2-7	--
MaxPool3d: 1-2	--
Sequential: 1-3	--
Conv3d: 2-8	13,856
BatchNorm3d: 2-9	64
ReLU: 2-10	--
Dropout: 2-11	--
Conv3d: 2-12	27,680
BatchNorm3d: 2-13	64
ReLU: 2-14	--
MaxPool3d: 1-4	--
Sequential: 1-5	--
Conv3d: 2-15	55,360
BatchNorm3d: 2-16	128
ReLU: 2-17	--
Dropout: 2-18	--
Conv3d: 2-19	110,656
BatchNorm3d: 2-20	128
ReLU: 2-21	--
MaxPool3d: 1-6	--
Sequential: 1-7	--
Conv3d: 2-22	221,312
BatchNorm3d: 2-23	256
ReLU: 2-24	--
Dropout: 2-25	--
Conv3d: 2-26	442,496
BatchNorm3d: 2-27	256
ReLU: 2-28	--
MaxPool3d: 1-8	--
Sequential: 1-9	--
Conv3d: 2-29	884,992
BatchNorm3d: 2-30	512
ReLU: 2-31	--
Dropout: 2-32	--
Conv3d: 2-33	1,769,728
BatchNorm3d: 2-34	512
ReLU: 2-35	--
ConvTranspose3d: 1-10	262,272
Sequential: 1-11	--
Conv3d: 2-36	884,864
BatchNorm3d: 2-37	256
ReLU: 2-38	--
Dropout: 2-39	--
Conv3d: 2-40	442,496
BatchNorm3d: 2-41	256
ReLU: 2-42	--
ConvTranspose3d: 1-12	65,600
Sequential: 1-13	--
Conv3d: 2-43	221,248
BatchNorm3d: 2-44	128
ReLU: 2-45	--
Dropout: 2-46	--
Conv3d: 2-47	110,656
BatchNorm3d: 2-48	128
ReLU: 2-49	--
ConvTranspose3d: 1-14	16,416
Sequential: 1-15	--
Conv3d: 2-50	55,328
BatchNorm3d: 2-51	64
ReLU: 2-52	--
Dropout: 2-53	--
Conv3d: 2-54	27,680
BatchNorm3d: 2-55	64
ReLU: 2-56	--
ConvTranspose3d: 1-16	4,112
Sequential: 1-17	--
Conv3d: 2-57	13,840
BatchNorm3d: 2-58	32
ReLU: 2-59	--
Dropout: 2-60	--
Conv3d: 2-61	6,928
BatchNorm3d: 2-62	32
ReLU: 2-63	--
Conv3d: 1-18	68
Total params: 5,648,772	
Trainable params: 5,648,772	
Non-trainable params: 0	

Figure 14: Chi tiết các lớp của mô hình

2.3. Quy trình huấn luyện

2.3.1 Hàm loss

Chọn hàm loss phù hợp cho một nhiệm vụ nhất định là nhiệm vụ quan trọng trong bất kỳ đào tạo học tập sâu nào. Đối với các nhiệm vụ phân đoạn ngữ nghĩa, hàm Dice Loss + Cross Entropy Loss hoạt động tốt

Tuy nhiên bài toán đang có các nhãn đang bị mất cân bằng về nhãn background vì vậy ta sẽ cân nhắc sử dụng Dice Loss + Focal Loss:

a. Dice Loss

- Dice Loss là một hàm loss thường được sử dụng trong các bài toán học máy, đặc biệt là trong lĩnh vực xử lý ảnh y tế (medical image segmentation) hoặc các bài toán phân đoạn (segmentation) nói chung. Nó dựa trên Dice Similarity Coefficient (DSC), một chỉ số đo lường mức độ tương đồng giữa hai tập hợp, thường là giữa nhãn dự đoán (predicted mask) và nhãn thực tế (ground truth).

- *Dice Similarity Coefficient (DSC):*

$$DSC = \frac{2(A \cap B)}{A \cup B}$$

$(A \cap B)$: Số phần tử chung giữa A và B (vùng giao nhau).

$A \cup B$: Tổng số phần tử của A và B (tổng diện tích của hai tập hợp).

- Giá trị của DSC nằm từ [0, 1]
 - DSC=0: không có sự chồng lấn nào
 - DSC=1: Hai tập hợp hoàn toàn trùng lặp
- Dice Loss:
$$Dice\ loss = 1 - DSC$$
 - Khi DSC cao (gần 1), Dice Loss sẽ thấp (gần 0), nghĩa là dự đoán của mô hình tốt.
 - Khi DSC thấp (gần 0), Dice Loss sẽ cao (gần 1), nghĩa là dự đoán của mô hình kém.
- Công thức cụ thể trong thực tế (đặc biệt với mạng nơ-ron) thường được viết lại để tránh chia cho 0 và phù hợp với dữ liệu xác suất (probability maps):

$$Dice\ loss = 1 - \frac{2 \sum i g_i * p_i + \epsilon}{\sum i p_i + \sum i g_i + \epsilon}$$

- p_i : Giá trị dự đoán tại pixel i (thường là xác suất từ 0 đến 1 sau hàm sigmoid hoặc softmax).
 - g_i : Giá trị thực tế tại pixel i (thường là 0 hoặc 1 trong nhãn nhị phân).
 - ϵ : Một hằng số nhỏ để tránh chia cho 0 khi cả p_i và g_i đều bằng 0.
- #### b. Focal Loss

Focal Loss là một hàm mất mát được thiết kế để giải quyết vấn đề mất cân bằng lớp (class imbalance) trong các bài toán phân loại, đặc biệt là trong các tác vụ như phát hiện đối tượng (object detection) hoặc phân đoạn ảnh (image

segmentation). Nó được giới thiệu bởi Tsung-Yi Lin và các đồng nghiệp trong bài báo "Focal Loss for Dense Object Detection" (2017). Trong các bài toán có dữ liệu mất cân bằng (ví dụ: nhiều mẫu nền, ít mẫu đối tượng), hàm mất mát thông thường như Cross-Entropy Loss có thể bị chi phối bởi các mẫu dễ phân loại (easy examples), dẫn đến việc mô hình không học tốt các mẫu khó (hard examples). Focal Loss giảm trọng số (weight) của các mẫu dễ phân loại và tập trung vào các mẫu khó bằng cách thêm một yếu tố điều chỉnh (modulating factor).

a. Công thức

$$FL(p_t) = \alpha_t(1 - p_t)^{\gamma} \log(p_t)$$

$$\text{Where: } p_t = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{if } y = 0 \end{cases}$$

$$\alpha_t = \begin{cases} \alpha & \text{if } y = 1 \\ 1 - \alpha & \text{if } y = 0 \end{cases}$$

2.3.2 Evaluation

Một metric phổ biến dùng để đánh giá hiệu suất của mô hình trong bài toán segmentation (phân đoạn ảnh) là IoU (Intersection over Union). IoU là một phép đo dùng để đánh giá mức độ chính xác của mô hình segmentation bằng cách so sánh vùng dự đoán (predicted region) với vùng thực tế (ground truth). Nó đo lường mức độ chồng lấp giữa hai vùng này.

$$IoU = \frac{\text{Intersection}}{\text{Union}} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

- Intersection: Vùng chồng lấp giữa phần dự đoán và ground truth (các pixel mà cả mô hình và ground truth đều đánh dấu là thuộc cùng một lớp).
- Union: Tổng vùng bao phủ của phần dự đoán và ground truth (tất cả pixel thuộc một trong hai vùng, trừ đi phần giao nhau để tránh đếm trùng).

Độ đo này nổi tiếng với bài toán Object Detection, tuy nhiên với bài toán này cách tính IoU sẽ khác so với bài OD tính toán dựa trên các vùng boundingbox. Đối với bài toán Segmentation làm việc với các pixel thay vì boundingbox, vì vậy ta cần tính tp, fp, fn, tn để tính được IoU.

- True Positives (tp): Số pixel được dự đoán đúng là thuộc lớp i (so với ground truth).
- False Positives (fp): Số pixel được dự đoán là lớp i, nhưng thực tế không phải (ground truth không phải lớp i).
- False Negatives (fn): Số pixel thực tế thuộc lớp i, nhưng mô hình dự đoán sai (không phải lớp i).

- True Negatives (tn): Số pixel không thuộc lớp i và mô hình cũng dự đoán đúng là không thuộc lớp i.

Tính IoU dựa trên tp, fp, fn, tn:

- Intersection = tp (số pixel dự đoán đúng).
- Union = tp+fp+ fn (tổng số pixel thuộc ít nhất một trong hai: dự đoán hoặc ground truth, nhưng không tính trùng tp).
- Công thức IoU cho một lớp:

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}} = \frac{tp}{tp + fp + fn}$$

2.3.3 Optimizer

AdamW (Adaptive Moment Estimation with Weight Decay regularization) là một biến thể của thuật toán Adam (Adaptive Moment Estimation). AdamW cải tiến Adam bằng cách xử lý tốt hơn weight decay, một kỹ thuật chính quy hóa giúp giảm overfitting thường được sử dụng trong các model deep learning hiện nay

- Adam: sự kết hợp của Gradient descent và RMSProp, sử dụng hai moment để điều chỉnh learning rate phù hợp
- AdamW: tách biệt weight decay khỏi quá trình cập nhật gradient, giúp cải thiện hiệu suất và tính ổn định khi huấn luyện mô hình

Công thức và cách hoạt động:

- Nhắc lại Adam:

- First moment (trung bình động của gradient):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t$$

Với: $\mathbf{g}_t$: gradient tại bước t, β_1 là hệ số (thường là 0.9)

- Second moment (trung bình động của gradient bình phương):

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

Với: $\mathbf{g}_t$: gradient tại bước t, β_2 là hệ số (thường là 0.999)

- Hiệu chỉnh bias (để tránh giá trị ban đầu nhỏ):

$$\mathbf{m_hat}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \mathbf{v_hat}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

- Cập nhật trọng số:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \alpha \frac{\mathbf{m_hat}_t}{\sqrt{\mathbf{v_hat}_t + \epsilon}}$$

Với: α : learning rate, ϵ (thường là 10^{-8})

- Weight decay trong Adam được tích hợp trực tiếp vào gradient:

$$g_t = \nabla_w \mathcal{L} + \lambda w$$

Với: λ : hệ số *weight decay* (thường là 0.09), tuy nhiên weight decay bị ảnh hưởng bởi momentum (v_t, m_t)

- AdamW: tách weigh decay ra khỏi gradient
 - Tính $v_t, m_t, v_hat_t, m_hat_t$, giống Adam nhưng chỉ dùng gradient của loss:

$$g_t = \nabla_w \mathcal{L}$$

- Cập nhật trọng số bằng 2 bước riêng biệt:

1. Cập nhật trọng số như ở Adam: $w_t = w_{t-1} - \alpha \frac{m_hat_t}{\sqrt{v_hat_t + \epsilon}}$

2. Áp dụng weight decay trực tiếp lên trọng số:

$$w_t = w_t' - \alpha \lambda w_{t-1}$$

- Ưu điểm của AdamW
 - Cải thiện chính quy hóa: Weight decay trong AdamW hoạt động hiệu quả hơn, giúp giảm overfitting tốt hơn so với Adam.
 - Hiệu suất tốt hơn: Nhiều thử nghiệm cho thấy AdamW thường dẫn đến kết quả tốt hơn trên các tác vụ học sâu, đặc biệt với các mô hình transformer hiện đại.
 - Tốc độ hội tụ nhanh: Kế thừa ưu điểm của Adam, AdamW vẫn giữ được tốc độ hội tụ nhanh nhờ learning rate thích ứng.

2.3.4 Quy trình training tổng quát

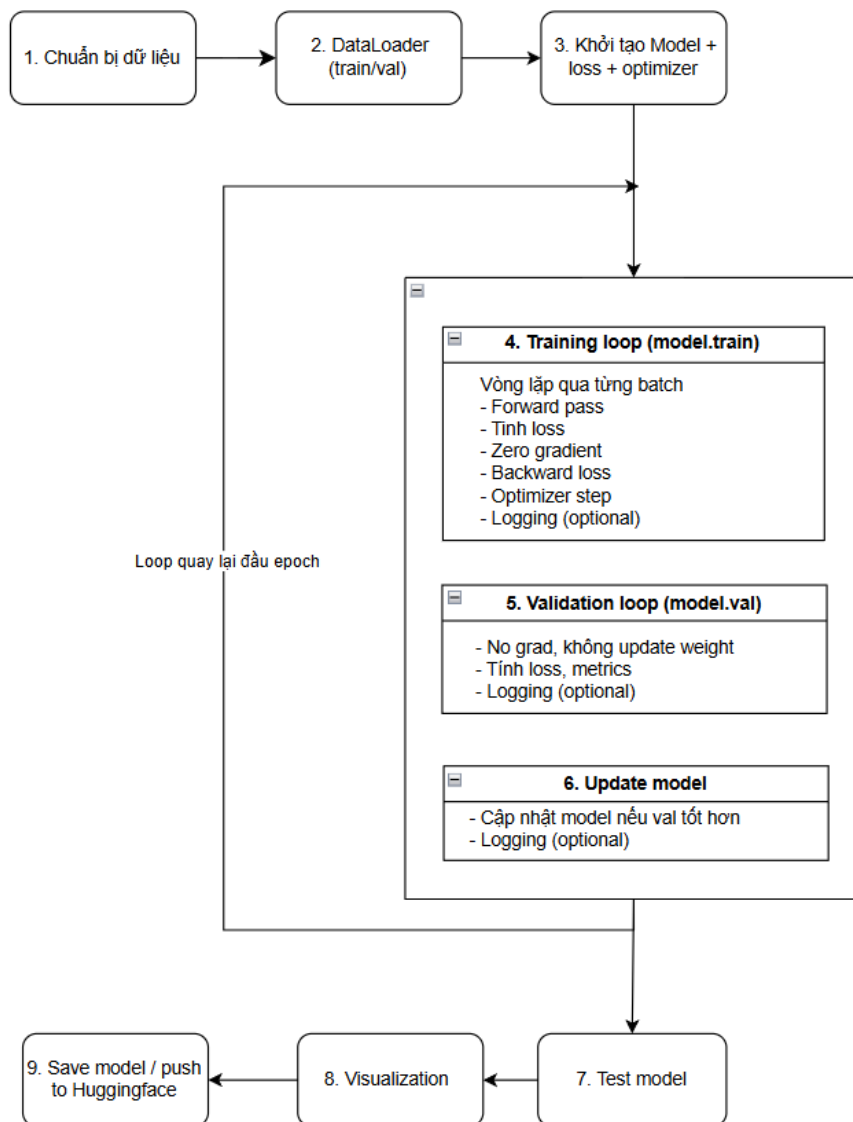


Figure 15: Quy trình huấn luyện mô hình

III. THỰC NGHIỆM

1. Môi trường thực nghiệm

Thực nghiệm được thực hiện trên nền tảng Kaggle, tận dụng GPU miễn phí để huấn luyện mô hình.

Cấu hình phần cứng

- GPU: 2 x NVIDIA T4 (16GB VRAM).
- RAM: 30GB
- Thời gian huấn luyện: Khoảng 5-6 giờ cho 25 epoch.

Phần mềm:

- Python: python version 3.11
- Thư viện: các thư viện được dùng sẽ có trong file requirements.txt của project
- Dữ liệu: tập BraTS2023 trên Kaggle

2. Phân tích mã nguồn

2.1 Xử lý dữ liệu

- Chuẩn hóa ảnh đầu vào (scale_image)
 - Input: Đường dẫn ảnh .nii (ảnh y tế 3D).
 - Xử lý:
 - Chuẩn hóa giá trị pixel bằng MinMaxScaler từ sklearn.
 - Đưa ảnh về dạng phẳng, scale, sau đó reshape về hình cũ.
 - Output: Ảnh đã chuẩn hóa.
- Lọc dữ liệu có quá ít vùng mask (ít hơn 1%):
 - if $(1 - (\text{counts}[0] / \text{counts.sum()})) > 0.01$
 - Nếu mask chứa quá ít vùng được đánh dấu (ít hơn 1%), ảnh đó bị bỏ qua để tiết kiệm tài nguyên.
- Stack nhiều modal ảnh MRI lại
 - `np.stack([temp_image_flair, temp_image_t1ce, temp_image_t2], axis=3)`
 - Gộp 3 loại ảnh (FLAIR, T1ce, T2) lại thành 1 tensor nhiều kênh giống ảnh RGB.
 - Cắt ảnh (crop) theo chiều không gian để giảm kích thước và tập trung vào ROI.
- Lưu ảnh và mask dưới dạng .npy
 - Tạo thư mục output nếu chưa tồn tại.
 - Chuyển mask sang dạng one-hot encoding (phục vụ segmentation).

- Lưu tensor ảnh và mask đã xử lý ra file .npy.

```
def scale_image(path_image):
    image = nib.load(path_image).get_fdata()
    image_resaped = image.reshape(-1, image.shape[-1])
    image_scaled = scaler.fit_transform(image_resaped).reshape(image.shape)
    return image_scaled

for idx in tqdm(
    range(446, len(t2_list)), desc="Preparing to stack, crop and save", unit="file"
):
    temp_mask = nib.load(mask_list[idx]).get_fdata()

    temp_mask = temp_mask[56:184, 56:184, 13:141]

    val, counts = np.unique(temp_mask, return_counts=True)

    # If a volume has less than 1% of mask, we simply ignore to reduce computation
    if (1 - (counts[0] / counts.sum())) > 0.01:

        temp_image_t1ce = scale_image(t1ce_list[idx])
        temp_image_t2 = scale_image(t2_list[idx])
        temp_image_flair = scale_image(flair_list[idx])

        temp_combined_images = np.stack(
            [temp_image_flair, temp_image_t1ce, temp_image_t2], axis=3
        )
        temp_combined_images = temp_combined_images[56:184, 56:184, 13:141]

        temp_mask = F.one_hot(torch.tensor(temp_mask, dtype=torch.long), num_classes=4)
        os.makedirs(IMAGE_OUTPUT_DIR, exist_ok=True)
        os.makedirs(MASK_OUTPUT_DIR, exist_ok=True)

        np.save(
            IMAGE_OUTPUT_DIR + f"/image_{idx}.npy",
            temp_combined_images,
        )
        np.save(
            MASK_OUTPUT_DIR + f"/mask_{idx}.npy",
            temp_mask,
        )
```

Figure 16: Quá trình xử lý dữ liệu

2.2. Xây dựng mô hình

- double_conv – Block tích chập cơ bản
 - Gồm: 2 lớp Conv3d + BatchNorm3d + ReLU, thêm Dropout.
 - Mục đích: học đặc trưng mạnh hơn, giảm overfitting tùy theo số kênh đầu ra (out_channels).
- UNETR – Kiến trúc mô hình
 - Contracting path (encoder): Gồm 4 tầng double_conv + MaxPool3d.
 - Bottleneck: gồm conv5 - tầng giao giữa encoder và decoder.
 - Expansive path (decoder):
 - ConvTranspose3d: Upsample.
 - torch.cat: Nối với tensor từ encoder (skip connection).
 - Sau đó là double_conv
 - Kết thúc: out_conv - Conv3d thành đầu ra số lớp = output_dim (ví dụ: 4 lớp mask).
- Hàm forward:
 - Encoder → Bottleneck → Decoder.
 - Skip connections giữ lại thông tin chi tiết từ encoder.

- Tạo ra output có cùng kích thước không gian với ảnh đầu vào.

```
def double_conv(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv3d(in_channels, out_channels, kernel_size=3, padding=1),
        nn.BatchNorm3d(out_channels), # Add BatchNorm3d after convolution
        nn.ReLU(inplace=True),
        nn.Dropout(0.1 if out_channels <= 32 else 0.2 if out_channels <= 128 else 0.3),
        nn.Conv3d(out_channels, out_channels, kernel_size=3, padding=1),
        nn.BatchNorm3d(out_channels), # Add BatchNorm3d after second convolution
        nn.ReLU(inplace=True)
    )
```

```
class UNETR(nn.Module):
    def __init__(self, image_size=128, input_dim=3, output_dim=4, d_model=128, patch_size=16, num_heads=2, dropout=0.1):
        super().__init__()

        # Contraction path
        self.conv1 = double_conv(in_channels=input_dim, out_channels=16) # [4, 16, 128, 128, 128]
        # self.tr1 = TransformerLayer(in_channels=16, image_size=image_size)
        self.pool1 = nn.MaxPool3d(kernel_size=2)

        self.conv2 = double_conv(in_channels=16, out_channels=32) # [1, 32, 64, 64, 64]
        # self.tr2 = TransformerLayer(in_channels=32, image_size=image_size//2)
        self.pool2 = nn.MaxPool3d(kernel_size=2)

        self.conv3 = double_conv(in_channels=32, out_channels=64)
        # self.tr3 = TransformerLayer(in_channels=64, image_size=image_size//4)
        self.pool3 = nn.MaxPool3d(kernel_size=2)

        self.conv4 = double_conv(in_channels=64, out_channels=128)
        # self.tr4 = TransformerLayer(in_channels=128, image_size=image_size//8)
        self.pool4 = nn.MaxPool3d(kernel_size=2)

        #BottleNeck
        self.conv5 = double_conv(in_channels=128, out_channels=256)

        #Decoder or Expansive path
        self.upconv6 = nn.ConvTranspose3d(in_channels=256, out_channels=128, kernel_size=2, stride=2)
        self.conv6 = double_conv(in_channels=256, out_channels=128)

        self.upconv7 = nn.ConvTranspose3d(in_channels=128, out_channels=64, kernel_size=2, stride=2)
        self.conv7 = double_conv(in_channels=128, out_channels=64)

        self.upconv8 = nn.ConvTranspose3d(in_channels=64, out_channels=32, kernel_size=2, stride=2)
        self.conv8 = double_conv(in_channels=64, out_channels=32)

        self.upconv9 = nn.ConvTranspose3d(in_channels=32, out_channels=16, kernel_size=2, stride=2)
        self.conv9 = double_conv(in_channels=32, out_channels=16)

        self.out_conv = nn.Conv3d(in_channels=16, out_channels=output_dim, kernel_size=1)
```

```

def forward(self, x):
    # Contracting path
    c1 = self.conv1(x)
    # c1 = self.tr1(c1)
    # print("c1: ", c1.shape)
    p1 = self.pool1(c1)

    c2 = self.conv2(p1)
    # c2 = self.tr2(c2)
    # print("c2: ", c2.shape)
    p2 = self.pool2(c2)

    c3 = self.conv3(p2)
    # c3 = self.tr3(c3)
    # print("c3: ", c3.shape)
    p3 = self.pool3(c3)

    c4 = self.conv4(p3)
    # c4 = self.tr4(c4)
    # print("c4: ", c4.shape)
    p4 = self.pool4(c4)

    # Bottleneck
    c5 = self.conv5(p4)

    # Expansive path
    u6 = self.upconv6(c5) # upscale
    u6 = torch.cat([u6, c4], dim=1) # skip connections along channel dim
    c6 = self.conv6(u6)

    u7 = self.upconv7(c6)
    u7 = torch.cat([u7, c3], dim=1)
    c7 = self.conv7(u7)

    u8 = self.upconv8(c7)
    u8 = torch.cat([u8, c2], dim=1)
    c8 = self.conv8(u8)

    u9 = self.upconv9(c8)
    u9 = torch.cat([u9, c1], dim=1)
    c9 = self.conv9(u9)

    outputs = self.out_conv(c9)

    return outputs

```

Figure 17: Xây dựng mô hình Unet3D

2.3 Huấn luyện mô hình

2.3.1 Training

- Chuẩn bị
 - Đặt model sang train()
 - Khởi tạo metric & loss recorder
- Lặp qua từng batch
 - Đưa data, target lên GPU
 - Forward → tính output
 - Tính loss, backprop và optimizer.step()
- Tính metric
 - Dùng argmax() để lấy label dự đoán
 - Tính IoU, Accuracy, update các recorder
- Hiện thị tiến trình
 - Dùng tqdm để show loss/IoU/acc
- Trả kết quả
 - Trả về loss, IoU, accuracy của toàn epoch

```

def train_one_epoch(
    model,
    loader,
    optimizer,
    num_classes,
    device="cpu",
    epoch_idx=800,
    total_epochs=10):

    model.train()

    loss_record = MeanMetric()
    metric_record = MeanMetric()
    acc_record = MulticlassAccuracy(num_classes=num_classes, average='macro') # Use macro-average accuracy

    loader_len = len(loader)

    with tqdm(total=loader_len, ncols=122) as tq:
        tq.set_description(f"Train :: Epoch: {epoch_idx}/{total_epochs}")

        for data, target in loader:
            tq.update(1)

            data, target = data.to(device).float(), target.to(device).float()

            optimizer.zero_grad()

            output_dict = model(data)

            target = target.argmax(dim=1) # Convert one-hot to class indices

            clsfy_out = output_dict # classifier head output

            loss = combined_loss(clsfy_out, target)

            # Calculate gradients w.r.t training parameters
            loss.backward()
            optimizer.step()

            # Detach for evaluation
            with torch.no_grad():
                pred_idx = clsfy_out.argmax(dim=1)

            # Calculate stats and IoU
            tp, fp, fn, tn = smp.metrics.get_stats(pred_idx, target, mode='multiclass', num_classes=num_classes)

            # Macro IoU and class-wise IoU
            metric_macro = smp.metrics.iou_score(tp, fp, fn, tn, reduction='macro')

            acc_record.update(pred_idx.cpu(), target.cpu())
            loss_record.update(loss.detach().cpu(), weight=data.shape[0])
            metric_record.update(metric_macro.cpu(), weight=data.shape[0])

            del tp, fp, fn, tn # Xóa ngay sau khi dùng
            del pred_idx, target, loss, metric_macro

        tq.set_postfix_str(s=f"Loss: {loss_record.compute():.4f}, IoU: {metric_record.compute():.4f}, Acc: {acc_record.compute():.4f}")

    epoch_loss = loss_record.compute()
    epoch_metric = metric_record.compute()
    epoch_acc = acc_record.compute()

    return epoch_loss, epoch_metric, epoch_acc

```

Figure 18: Hàm training cho một epoch

2.3.2 Validation

- Chuyển model sang eval()
- Lặp qua từng batch
 - Không tính gradient (no_grad)
 - Forward → Output → Dự đoán label
 - Tính loss, IoU, accuracy
 - Ghi nhận từng batch vào recorder

- Tổng hợp kết quả
 - Trung bình loss, IoU, acc toàn epoch
- Trả về:
 - valid_epoch_loss, valid_epoch_metric, valid_epoch_acc

```
# Validation function, logging macro IoU and per-class IoU.
def validate(
    model,
    loader,
    device,
    num_classes,
    epoch_idx,
    total_epochs
):
    model.eval()

    loss_record = MeanMetric()
    metric_record = MeanMetric()
    acc_record = MulticlassAccuracy(num_classes=num_classes, average='macro')

    loader_len = len(loader)

    with tqdm(total=loader_len, ncols=122) as tq:
        tq.set_description(f"Valid :: Epoch: {epoch_idx}/{total_epochs}")

        for data, target in loader:
            tq.update(1)

            data, target = data.to(device).float(), target.to(device).float()

            with torch.no_grad():
                output_dict = model(data)

            clsfy_out = output_dict
            target = target.argmax(dim=1) # Convert one-hot to class indices

            loss = combined_loss(clsfy_out, target)
            pred_idx = clsfy_out.argmax(dim=1)

            tp, fp, fn, tn = smp.metrics.get_stats(pred_idx, target, mode='multiclass', num_classes=num_classes)

            # Macro IoU
            metric_macro = smp.metrics.iou_score(tp, fp, fn, tn, reduction='macro')

            acc_record.update(pred_idx.cpu(), target.cpu())
            loss_record.update(loss.cpu(), weight=data.shape[0])
            metric_record.update(metric_macro.cpu(), weight=data.shape[0]) #data.shape = batch

            del tp, fp, fn, tn # Xóa ngay sau khi dùng
            del pred_idx, target, loss, metric_macro

    valid_epoch_loss = loss_record.compute()
    valid_epoch_metric = metric_record.compute()
    valid_epoch_acc = acc_record.compute()

    return valid_epoch_loss, valid_epoch_metric, valid_epoch_acc
```

Figure 19: Hàm validation cho một epoch

3. Kết quả

Các thông số gồm accuracy, loss, metric IoU trên cả tập train và val được hiển thị như trong ảnh.

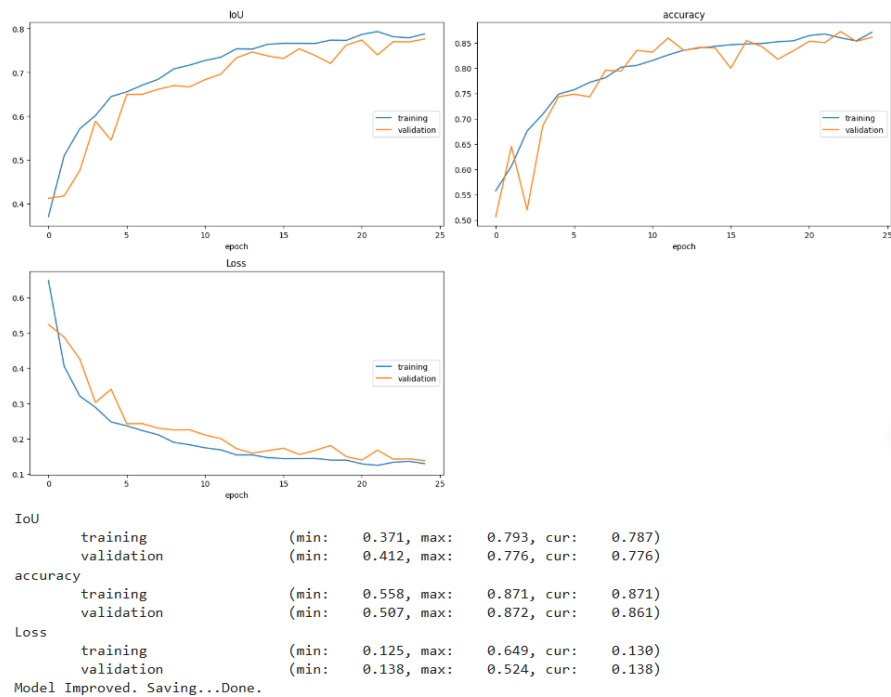


Figure 20: Kết quả sau khi huấn luyện

	Accuracy	Loss	IoU
Training	0.871	0.130	0.787
Validation	0.861	0.138	0.776

- Bảng thử nghiệm trên các thông số khác nhau:

Hyperparam	Loss function	Config	Val IoU
init_lr=1e-3, epochs=30, Multi Step Lr with gamma = 0.1 @ [60,90]	Dice Loss + Focal Loss	No Batch Norm	0.746
init_lr = 1e-3, epochs=25, Cosine Annealing	Dice Loss + Focal Loss	No BatchNorm	0.723
init_lr = 1e-3, epochs = 25	Dice Loss + Focal Loss	BatchNorm	0.787
init_lr = 1e-4, epochs = 30	Dice + Focal Loss + Tverskey	No BatchNorm	0.75

- Inference trên 1 vài mẫu dữ liệu của tập test:

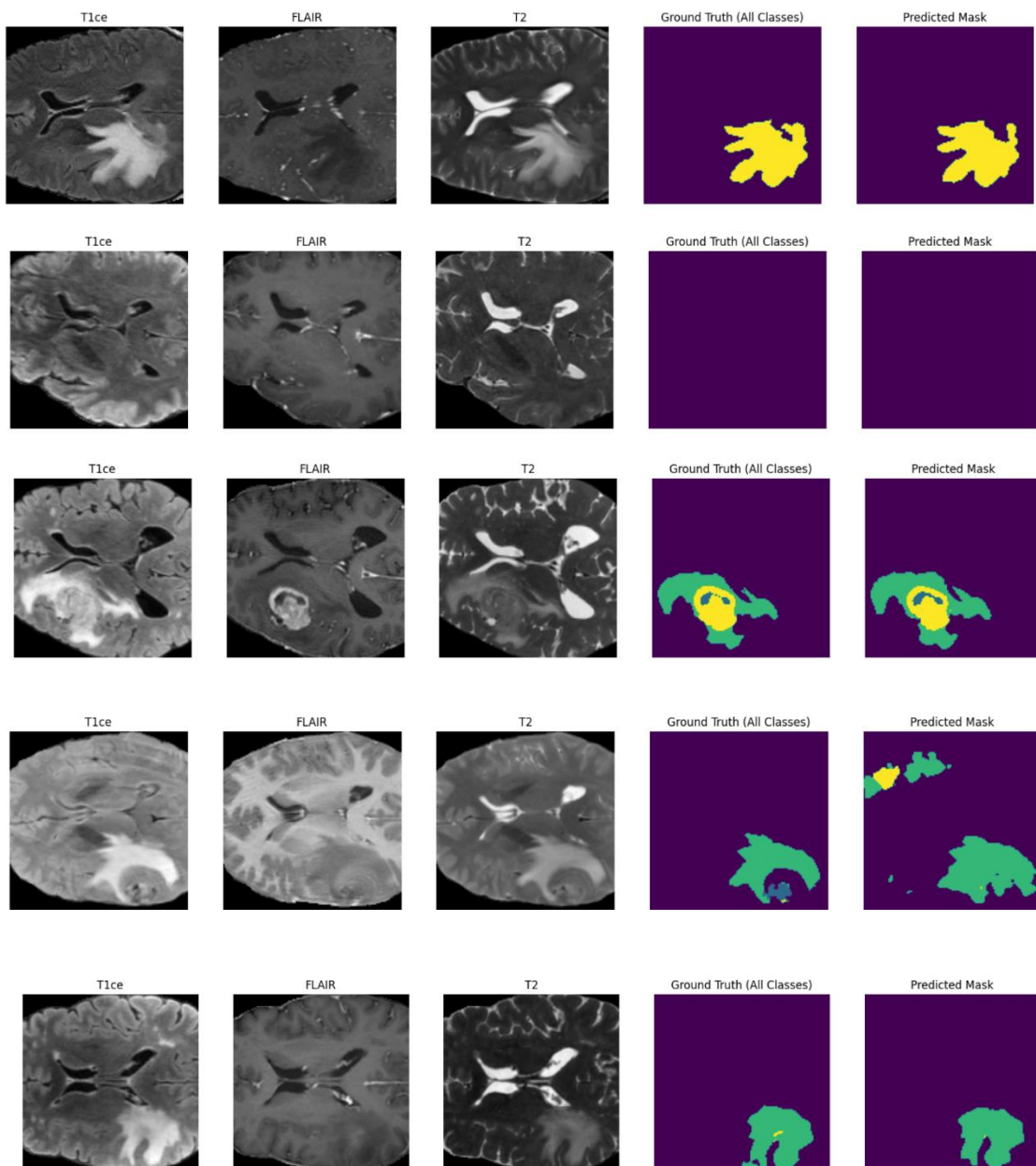


Figure 21: Kết quả sau khi mô hình được huấn luyện

4. Triển khai hệ thống

- Sơ đồ hệ thống
 - Lưu checkpoint của model để inference

- Sử dụng Streamlit để triển khai giao diện một cách nhanh chóng

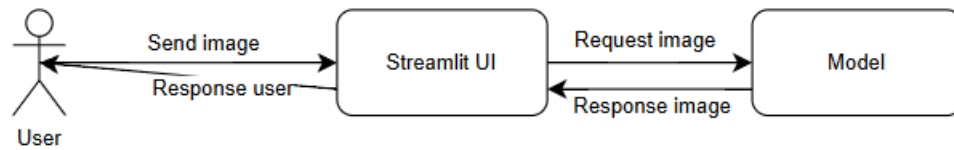


Figure 22: Kiến trúc hệ thống

- Kết quả trên hệ thống:

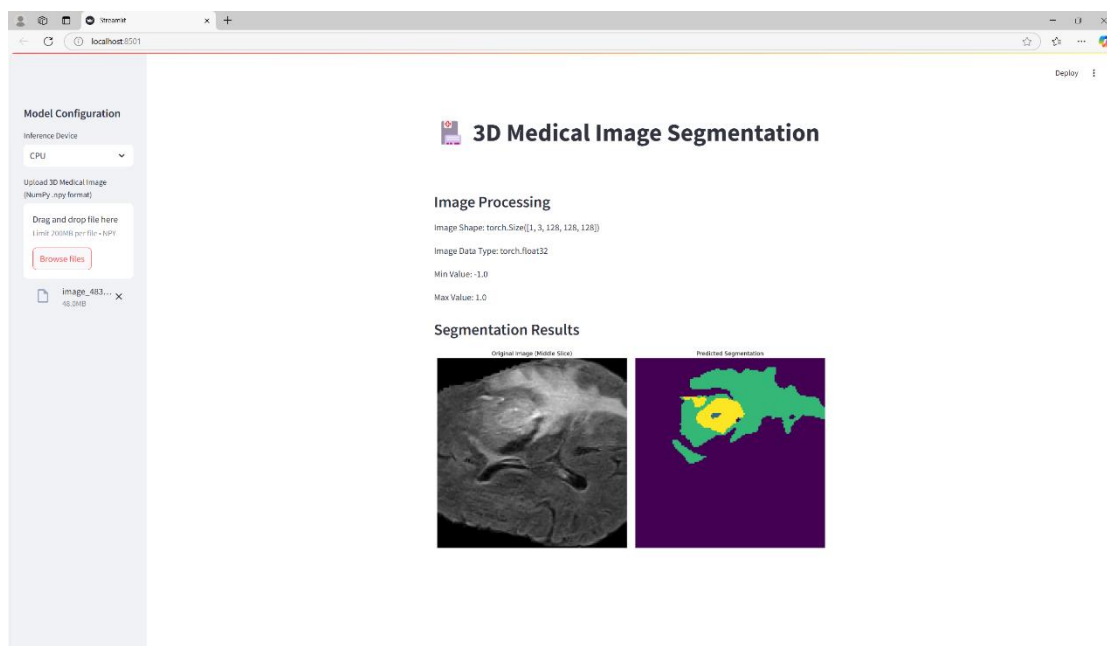


Figure 23: Giao diện của hệ thống

5. Phân tích

Ưu điểm:

- Kiến trúc CNN cùng với mô hình Unet đặc trưng cho bài toán segmentation đã giúp mô hình nắm bắt được đặc trưng của ảnh, phân đoạn tương đối các vùng bị bệnh
- Cơ chế skip connection của Unet giúp giữ lại chi tiết cục bộ đảm bảo độ chính xác cho các vùng nhỏ như lõi khối u
- Chỉ 25 epoch với 1 model được training từ đầu đã đạt được kết quả tương đối
- Độ ổn định của mô hình được cải thiện nhờ kỹ thuật tăng cường dữ liệu và sự kết hợp của các hàm loss

Hạn chế

- Hạn chế về phần cứng: tuy sử dụng 2 GPU T4 nhưng dữ liệu lớn dẫn đến việc training mất nhiều thời gian
- Dữ liệu lớn (70 – 80 GB): yêu cầu ổ đĩa có dung lượng lớn để lưu dữ liệu
- Mô hình chưa được phát triển hết: chưa kết hợp được với Transformer – một kiến trúc mạnh mẽ cho thời đại Deep learning hiện nay do phần cứng hạn chế

KẾT LUẬN

Trong khuôn khổ đề án, nhóm đã tiến hành nghiên cứu, triển khai và đánh giá hiệu quả của mô hình 3D U-Net cho bài toán phân đoạn khối u não trên ảnh MRI. Đây là một hướng tiếp cận hiện đại, cho phép tận dụng được thông tin không gian ba chiều của dữ liệu thể tích, nhằm cải thiện độ chính xác trong việc xác định vùng tổn thương so với các phương pháp 2D truyền thống.

Kết quả thực nghiệm cho thấy mô hình 3D U-Net có khả năng phân đoạn các vùng u não ở mức độ tương đối tốt, dù vẫn còn một số hạn chế nhất định về độ chính xác. Nguyên nhân chủ yếu đến từ giới hạn về phần cứng tính toán và cấu hình mô hình chưa đủ lớn để khai thác toàn diện tiềm năng của kiến trúc. Tuy nhiên, đây là tiền đề quan trọng để tiếp tục phát triển các mô hình sâu hơn, mạnh hơn trong tương lai.

Thông qua đề án, nhóm không chỉ đạt được mục tiêu xây dựng một mô hình AI ứng dụng trong y tế, mà còn tích lũy thêm kinh nghiệm thực tế trong việc xử lý dữ liệu ảnh y tế, huấn luyện mô hình học sâu và đánh giá kết quả. Hướng phát triển tiếp theo có thể bao gồm tối ưu hóa mô hình, áp dụng kỹ thuật tăng cường dữ liệu (data augmentation), thử nghiệm các kiến trúc lai (như kết hợp Attention hoặc Transformer), cũng như đánh giá mô hình trên tập dữ liệu lớn và đa dạng hơn.

Nhóm kỳ vọng rằng kết quả của đề tài sẽ là bước khởi đầu hữu ích trong hành trình ứng dụng trí tuệ nhân tạo vào hỗ trợ chẩn đoán y tế, góp phần nâng cao hiệu quả trong công tác phát hiện và điều trị sớm các bệnh lý liên quan đến não bộ.

TÀI LIỆU THAM KHẢO

<https://tamanhhospital.vn/chup-cong-huong-tu-mri/>

<https://www.kaggle.com/datasets/shakilrana/brats-2023-adult-glioma>

<https://arxiv.org/abs/1505.04597>

<https://streamlit.io/>

<https://learnopencv.com/3d-u-net-brats/>